EmoSense — Machine-Learning Documentation

Notebook ser_report_notebook.ipynb explained: data, preprocessing, models, evaluation, export

Contents of this document

- 01-ml/README.md
- 01-ml/01-data-and-labels.md
- 01-ml/02-preprocessing-augmentation.md
- 01-ml/03-models.md
- 01-ml/04-training-evaluation.md
- 01-ml/05-export-and-realtime.md

Generated from the Markdown files in the docs folder (the Markdown files remain the source of truth).

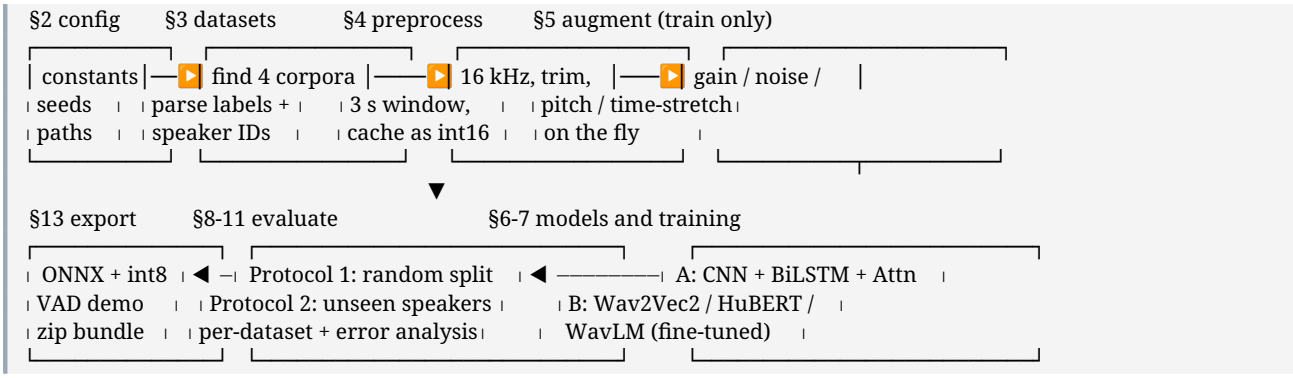
1. The machine-learning part

Notebook: `ser_report_notebook.ipynb` (generated from `tools/report_notebook.py`). It runs on a free **Kaggle GPU (T4)** and produces: the trained models, all evaluation numbers, the report figures and the `model.onnx` file used by the cloud API.

Purpose

Turn 3-second audio clips into one of **seven emotions** (angry, disgust, fear, happy, neutral, sad, surprise) and measure honestly how well that works on **new people**.

The workflow in one picture



Map: notebook section → documentation page

Notebook §	Content	Page
1	Introduction, objectives, design decisions	(markdown only; summarised in ../README.md)
2	Environment and configuration	01-data-and-labels.md
3	Datasets and label harmonisation	01-data-and-labels.md
4–5	Pre-processing and augmentation	02-preprocessing-augmentation.md
6	Model architectures	03-models.md
7–11	Training, evaluation protocols, analysis	04-training-evaluation.md
12	Auto-generated final results and abstract	04-training-evaluation.md
13	Export, quantisation, real-time demo	05-export-and-realtime.md
14	Conclusion, limitations, references	(markdown only)

Why this design (components at a glance)

Component	What	Why
`QUICK_RUN` switch	Runs the whole pipeline on ~600 clips and 1 epoch	Catches bugs in 15 minutes instead of after 6 hours of GPU time
Results saved to JSON after every model/fold	Resumable	Kaggle sessions can stop; nothing is lost
Cached waveforms as `int16` `.npy`	Decode audio once, memory-map afterwards	Loading 12 000 WAV files every epoch would be far too slow

Component	What	Why
One generic `train_model` for all models	Same optimiser, loss, schedule	Differences in results come from the model, not from tuning tricks
Validation set for model selection, test set used once	Prevents optimistic bias	A test set used for choices is no longer a test set
Figures saved as PNG (`show()`)	Ready for the project report	No re-plotting needed

How to explain it to the mentor in 60 seconds

> *"We combined four emotional-speech datasets into one seven-class problem. Audio is cleaned and cut to 3 seconds, and we augment only the training data to avoid leakage. We trained a small CNN-BiLSTM-attention network and three large pre-trained speech transformers, and evaluated them twice: once with a random split, which is the usual headline number, and once with speaker-independent cross-validation, which tells us how the system behaves for a new user. The best speaker-independent model was exported to ONNX and deployed."*

1a. Configuration, datasets and labels (notebook §2–3)

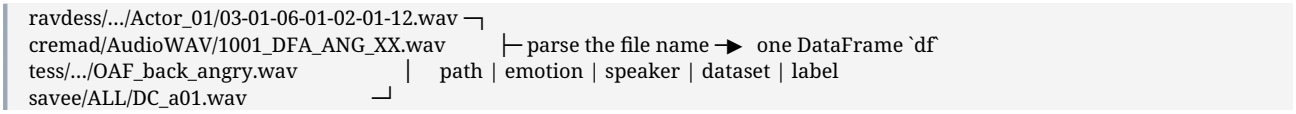
Purpose

Collect all settings in one place, find the four datasets, read each file name to get **the emotion label and the speaker**, and make all datasets share one label set.

Why this component

Piece	What it does	Why we need it	Rejected alternative
Single configuration cell	All constants (sample rate, epochs, learning rates …) at the top	Reproducible; easy to change one number	Magic numbers spread over many cells
`find_root`	Finds a dataset folder on Kaggle or locally	Kaggle mounts datasets at different paths than Colab	Hard-coded paths break on another machine
File-name parsing	The emotion and speaker are encoded in file names	The datasets have no label files	Manual labelling (impossible for 12 000 clips)
Speaker ID per clip	Needed by Protocol 2	To keep a speaker entirely in train *or* test	Random split only (leaks speakers)
Common 7-label set	`EMOTIONS` list	Four corpora use different label sets	Training one model per corpus
Class weights	Give rare classes more influence	*Surprise* is rare	Throwing data away to balance

Diagram: from folders to one table



Line-by-line walkthrough

Imports

```
import os, sys, json, math, random, time, glob, re, copy, itertools, shutil, warnings
```

Standard-library tools: `os`/`glob` for files and folders, `json` to save results, `math` for π in the spectrogram, `random` for seeds, `time` for timing, `re` for pattern matching file names, `itertools` for combinations (ensemble), `shutil` for copying/zipping.

```
import numpy as np
import pandas as pd
import torch, torchaudio
import torch.nn as nn
import torch.nn.functional as F
```

`numpy` = numerical arrays (audio is an array), `pandas` = tables (our `df`), `torch` = the deep-learning library, `torchaudio` = audio helpers (mel filters, SpecAugment), `nn` = layers, `F` = functions such as padding and convolution.

```
import librosa, soundfile as sf
```

`librosa` loads/resamples audio and trims silence; `soundfile` reads/writes WAV files.

```
from sklearn.model_selection import train_test_split, GroupKFold, GroupShuffleSplit
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
from sklearn.utils.class_weight import compute_class_weight
```

scikit-learn provides the **splitting** tools (random split, speaker-aware folds), the **metrics**, and the **class-weight** formula.

```
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid", context="notebook")
```

Hide noisy library warnings; give all figures one clean style.

Configuration

```
QUICK_RUN   = False      # True -> tiny subset & 1 epoch: checks the whole pipeline in ~15 min
```

A safety switch: set it to `True` first to verify nothing crashes before spending hours of GPU time.

```
SEED       = 42
```

The random seed. Fixing it makes splits and initial weights repeatable.

```
SR         = 16_000                # target sample rate (Hz) - what all pretrained speech models expect
DUR        = 3.0                   # fixed analysis window (s)
N_SAMPLES  = int(SR * DUR)         # 48 000 samples
```

`SR` is the sample rate. `DUR` is the window length. `N_SAMPLES` = 16 000 × 3 = 48 000 numbers per clip – **every model input has exactly this length**. *Why 3 s?* Most dataset clips are 2–4 s; a fixed length makes batching simple and gives constant latency in the app.

```
TOP_DB     = 30                    # silence threshold (dB below peak) for librosa.effects.trim
```

Anything quieter than 30 dB below the loudest part at the start or end is treated as silence and removed.

```
EMOTIONS   = ["angry", "disgust", "fear", "happy", "neutral", "sad", "surprise"]
LABEL2ID   = {e: i for i, e in enumerate(EMOTIONS)}
N_CLASSES  = len(EMOTIONS)
```

The **7 classes**. `LABEL2ID` turns a name into a number (`"angry"` → 0, …, `"surprise"` → 6) because networks output numbers. The order here is the same order the API returns probabilities.

```
BATCH_A, LR_A = 64, 2e-3
P1_EPOCHS_A, P2_EPOCHS_A = 30, 20
```

Model A (small CNN): batch of 64 clips, learning rate 0.002, 30 epochs for Protocol 1, 20 for Protocol 2. A small network trains quickly and tolerates a larger learning rate.

```
HF_IDS = {                         # name -> Hugging Face model id
   "Wav2Vec2-base":   "facebook/wav2vec2-base",
   "HuBERT-base":     "facebook/hubert-base-ls960",
   "WavLM-base-plus": "microsoft/wavlm-base-plus",
}
```

The three pre-trained transformers we compare, by their Hugging Face names (downloaded automatically; this is why Kaggle's *Internet* switch must be on).

```
BATCH_B, LR_B_BACKBONE, LR_B_HEAD = 16, 5e-5, 1e-3
P1_EPOCHS_B, P2_EPOCHS_B = 12, 6        # P2 uses fewer epochs because it is repeated for 5 folds
```

Model B: smaller batch (16) because the transformer needs more GPU memory; **a tiny learning rate (0.00005) for the pre-trained body** so we gently adapt it without destroying what it learned, and a larger rate (0.001) for the new, randomly initialised head. Protocol 2 repeats training 5 times, so each run uses fewer epochs to fit the time budget.

```
N_FOLDS     = 5
P2_TRANSFORMERS = 1                  # how many of the best transformers (by P1 *validation* acc) also get Protocol 2
ENABLE_ENSEMBLE = False              # optional: average transformer probabilities (weights/subset chosen on validation)
NUM_WORKERS = min(4, os.cpu_count() or 2) # CPU workers for on-the-fly augmentation
```

5 folds for speaker-independent testing; only the best transformer (chosen on **validation** accuracy, never the test set) gets this expensive protocol; the ensemble is optional and off; the number of CPU workers that augment audio while the GPU trains (Kaggle has 4 CPU cores).

```
if QUICK_RUN:
    P1_EPOCHS_A = P2_EPOCHS_A = P1_EPOCHS_B = P2_EPOCHS_B = 1
```

In quick mode every training run lasts a single epoch.

```
IN_KAGGLE = os.path.isdir("/kaggle/input")
IN_COLAB  = "google.colab" in sys.modules
WORK_DIR  = "/kaggle/working/ser_report" if IN_KAGGLE else ("/content/ser_report" if IN_COLAB else "./ser_report")
```

Detect where the notebook runs and pick a folder for outputs. On Kaggle, `/kaggle/working` is the folder whose content you can download afterwards.

```
FIG_DIR   = os.path.join(WORK_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)
TAG = "quick" if QUICK_RUN else "full"
RESULTS_PATH = os.path.join(WORK_DIR, f"results_{TAG}.json")
```

Folder for figures, a tag so quick-run and full-run files never mix, and the path of the results file.

```
def seed_everything(seed=SEED):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
seed_everything()
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Seed all random generators (Python, NumPy, PyTorch CPU and GPU) so a rerun gives the same numbers; choose the GPU when present.

```
RESULTS = json.load(open(RESULTS_PATH)) if os.path.exists(RESULTS_PATH) else {"p1": {}, "p2": {}}
def save_results(): json.dump(RESULTS, open(RESULTS_PATH, "w"))
```

The **resumable results store**: if the file exists we reload what was finished; `save_results()` is called after every model or fold. If Kaggle disconnects, a rerun skips finished work.

```
def show(name):
    """Save the current matplotlib figure as a report-ready PNG (200 dpi) and display it."""
    plt.savefig(os.path.join(FIG_DIR, name + ".png"), dpi=200, bbox_inches="tight"); plt.show()
```

Every figure is saved as a high-resolution PNG for the report *and* shown in the notebook.

Finding the datasets

```
def find_root(slug):
    cands = [f"/kaggle/input/{slug}", f"./{slug}"] + glob.glob(f"/kaggle/input/datasets/*/{slug}") + glob.glob(f"/kaggle/input/*/{slug}")
    return next((c for c in cands if os.path.isdir(c)), None)
```

Build a list of places where a dataset folder might be and return the first that exists (or `None`). Kaggle has used several mount layouts over time, so we try them all.

```
def locate_all():
```

```
    return dict(Ravdess=find_root("ravdess-emotional-speech-audio"), Crema=find_root("cremad"),
                Savee=find_root("surrey-audiovisual-expressed-emotion-savee"), Tess=find_root("toronto-emotional-speech-set-tess"))
```

`locate_all` calls `find_root` once per dataset and returns a small dictionary `{"Ravdess": path-or-None, ...}`. The folder names are the Kaggle dataset names (slugs). On Colab/local machines, if any entry is `None` the notebook downloads the data with the Kaggle API (`opendatasets`) and calls `locate_all()` again.

```
assert all(roots.values()), f"Missing dataset(s): {[k for k, v in roots.items() if not v]} - use 'Add Input' on Kaggle"
```

Stop immediately with a clear message if any of the four datasets is missing, instead of failing later with a confusing error.

```
_rav = glob.glob(os.path.join(roots["Ravdess"], "**", "audio_speech_actors_01-24"), recursive=True)
```

The RAVDESS download also contains **song** recordings. We search for the folder `audio_speech_actors_01-24` and use only speech.

```
def _wavs(folder):
    return sorted(p for p in glob.glob(os.path.join(folder, "**", "*"), recursive=True) if p.lower().endswith(".wav"))
```

List every `.wav` file below a folder, in sorted (deterministic) order.

Label maps

```
RAV_MAP   = {1: "neutral", 2: "neutral", 3: "happy", 4: "sad", 5: "angry", 6: "fear", 7: "disgust", 8: "surprise"}   # 2 = calm -> neutral
```

RAVDESS stores the emotion as a number. **Code 2 means "calm"; we map it to "neutral"** (the other datasets have no calm class). This is the deliberate decision the brief required to be documented.

```
CREMA_MAP = {"ANG": "angry", "DIS": "disgust", "FEA": "fear", "HAP": "happy", "NEU": "neutral", "SAD": "sad"}
SAVEE_MAP = {"a": "angry", "d": "disgust", "f": "fear", "h": "happy", "n": "neutral", "sa": "sad", "su": "surprise"}
```

CREMA-D uses three-letter codes (and has no surprise); SAVEE uses one or two letters.

```
TESS_MAP  = {"angry": "angry", "disgust": "disgust", "fear": "fear", "happy": "happy", "neutral": "neutral",
             "sad": "sad", "ps": "surprise", "surprise": "surprise"}
```

TESS writes the emotion as a word; `ps` = "pleasant surprise" becomes *surprise*.

Reading each file name

```
rows = []   # (path, emotion, speaker, dataset, was_calm)
for p in _wavs(Ravdess):
    f = os.path.basename(p)[:-4].split("-")
    if len(f) == 7 and int(f[2]) in RAV_MAP:
        rows.append((p, RAV_MAP[int(f[2])], f"RAVDESS_{f[6]}", "RAVDESS", int(f[2]) == 2))
```

`os.path.basename(p)[:-4]` removes the folder and the `.wav` ending, `.split("-")` cuts `03-01-06-01-02-01-12` into 7 fields. Field 3 (`f[2]`) is the emotion code; field 7 (`f[6]`) is the **actor number** = **speaker**. The last value remembers whether the clip was "calm", only to print how many were merged.

```
for p in _wavs(Crema):
    f = os.path.basename(p)[:-4].split("_")
    if len(f) >= 3 and f[2] in CREMA_MAP:
        rows.append((p, CREMA_MAP[f[2]], f"CREMAD_{f[0]}", "CREMA-D", False))
```

`1001_DFA_ANG_XX` → speaker `1001`, emotion `ANG`.

```
for p in _wavs(Tess):
    f = os.path.basename(p)[:-4].split("_")
    if len(f) >= 3 and f[-1].lower() in TESS_MAP:
```

```
        rows.append((p, TESS_MAP[f[-1].lower()], f"TESS_{f[0].upper()}", "TESS", False))
```

`OAF_back_angry` → speaker `OAF` (older female) and emotion = last token.

```
for p in _wavs(Savee):
    f = os.path.basename(p)[:-4].split("_")
    m = re.match(r"[a-z]+", f[1]) if len(f) > 1 else None
    if m and m.group() in SAVEE_MAP:
        rows.append((p, SAVEE_MAP[m.group()], f"SAVEE_{f[0].upper()}", "SAVEE", False))
```

`DC_a01` → speaker `DC`; the regular expression `[a-z]+` takes the letters before the number (`a`, `sa`, …) as the emotion code.

Building the table

```
df = pd.DataFrame(rows, columns=["path", "emotion", "speaker", "dataset", "was_calm"])
df["label"] = df.emotion.map(LABEL2ID)
assert df.dataset.nunique() == 4, "all four datasets must be present"
```

Turn the list into a table and add the numeric label. The `assert` guarantees the requirement "never fewer than 4 datasets".

```
if QUICK_RUN:   # small stratified subset for the smoke test
    df = df.sample(frac=1, random_state=SEED).groupby("emotion").head(90)
df = df.reset_index(drop=True)
```

In quick mode shuffle, then keep at most 90 clips per emotion; finally renumber rows 0…N-1 (many later arrays are indexed by these positions).

Looking at the data (EDA)

```
print(pd.crosstab(df.dataset, df.emotion, margins=True))
```

A table counting clips per dataset and emotion – shows imbalance (e.g. CREMA-D has zero *surprise*).

```
cw = compute_class_weight("balanced", classes=np.arange(N_CLASSES), y=df.label.values)
print("Inverse-frequency class weights:", dict(zip(EMOTIONS, cw.round(2))))
```

The formula `w_c = N / (K · n_c)`: a class with fewer clips gets a larger weight. Here it is only printed for inspection; during training the weights are recomputed from the **training fold only** (so test information never leaks).

Things to tell the mentor

- **Speaker IDs were extracted on purpose** – they are the foundation of the speaker-independent evaluation.
- **calm → neutral** is an explicit, documented decision.
- **`assert` statements** document assumptions and fail early.
- The results file makes the notebook **resumable** on a free GPU platform.

1b. Pre-processing and augmentation (notebook §4–5)

Purpose

1. Turn every audio file into **exactly the same format**: 16 kHz, mono, speech only, 3 seconds long.
2. Cache the result so training is fast.
3. Create extra, slightly modified training examples (**augmentation**) — *only for the training data*.

Why this component

Piece	What	Why	Rejected alternative
Resample to 16 kHz mono	Same sample rate for all files	The pretrained transformers were trained on 16 kHz; datasets differ (22 kHz, 44 kHz …)	Feeding mixed rates: wrong results
Trim silence	Remove quiet start/end	Silence carries no emotion and wastes the 3 s window	Keeping silence: model learns recording-room noise
Fixed 3 s window	Crop or pad	Batching needs equal lengths; constant latency in the app	Variable lengths: complex batching, variable latency
Cache as `int16` `.npy`	Decode once	12 000 files decoded every epoch would dominate run time	Re-reading WAVs each epoch
Augmentation in `Dataset`	Random changes on the fly	More variety → less over-fitting; robust to microphones/noise	Pre-augmenting before splitting = **data leakage**
`train`-only augmentation	Val/test stay untouched	Evaluation must reflect real, unmodified audio	Augmenting everything

Diagram: the one function used in training *and* in the API

```
raw audio file
   │ librosa.load(sr=16000, mono=True)      → resample + mix to mono
   ▼
float array of any length
   │ librosa.effects.trim(top_db=30)        → cut silent edges
   ▼
speech-only array
   ╵ centre-crop if > 3 s, zero-pad if < 3 s
   ▼
exactly 48 000 samples ──▶ model
```

The backend re-implements the same steps (see ../02-backend/README.md) so that the app's audio is treated exactly like the training audio. This is called **train/serve parity**.

Line-by-line walkthrough

`to_fixed`

```
def to_fixed(y, n=N_SAMPLES):
```

`y` is the audio as a NumPy array; `n` is the target length (48 000).

```
    if len(y) == 0:
        return np.zeros(n, dtype=np.float32)
```

Safety net: a broken or empty file becomes a silent 3 s clip instead of crashing.

```
    yt, _ = librosa.effects.trim(y, top_db=TOP_DB)
```

Removes leading and trailing parts that are more than 30 dB quieter than the loudest part. It returns the trimmed audio and the index range (ignored, hence `_`).

```
    if len(yt) < 0.1 * SR:              # trimming removed (almost) everything -> keep the original
        yt = y
```

If trimming left less than 0.1 s (a very quiet or whispery clip) we keep the original rather than feeding an almost empty signal.

```
    if len(yt) >= n:                    # centre crop
        s = (len(yt) - n) // 2
        yt = yt[s:s + n]
```

Longer than 3 s: take the **middle** 3 s (`s` = how much to skip at the start). The middle usually contains the strongest emotional content.

```
    else:                               # zero-pad at the end
        yt = np.pad(yt, (0, n - len(yt)))
    return yt.astype(np.float32)
```

Shorter than 3 s: add zeros at the end. Return 32-bit floats because that is what the network expects.

`load_clip` and the cache

```
def load_clip(path):
    y, _ = librosa.load(path, sr=SR, mono=True)          # resample to 16 kHz mono
    return (np.clip(to_fixed(y), -1, 1) * 32767).astype(np.int16)    # int16 cache saves 50 % memory
```

Load + resample + mix to mono in one call; run `to_fixed`; clip values into [−1, 1]; multiply by 32767 and store as 16-bit integers (half the memory of 32-bit floats, no audible loss).

```
WAV_PATH, META_PATH = os.path.join(WORK_DIR, f"waves_{TAG}.npy"), os.path.join(WORK_DIR,
f"meta_{TAG}.csv")
if os.path.exists(WAV_PATH) and os.path.exists(META_PATH) and len(pd.read_csv(META_PATH)) == len(df):
    print("loading cached waveforms")
```

If a cache from an earlier run exists *and* has the same number of rows as the current table, reuse it.

```
    waves = Parallel(n_jobs=-1, batch_size=32)(delayed(load_clip)(p) for p in df.path)
    np.save(WAV_PATH, np.stack(waves)); df.to_csv(META_PATH, index=False)
```

Otherwise process all files **in parallel on every CPU core** (`joblib`), stack them into one `(N, 48000)` array and save it together with the metadata table.

```
WAVES = np.load(WAV_PATH, mmap_mode="r")          # (N, 48000) int16, memory-mapped
LABELS, SPEAKERS, DATASETS = df.label.values, df.speaker.values, df.dataset.values
```

`mmap_mode="r"` means the array stays on disk and is read page by page, so RAM stays small. We also keep the labels, speakers and dataset names as plain arrays; **all later splitting works with row numbers into these arrays**.

The mel spectrogram figure

```
    S = librosa.power_to_db(librosa.feature.melspectrogram(y=y, sr=SR, n_fft=1024, hop_length=256, n_mels=64), ref=np.max)
```

A *mel spectrogram* is a picture: x = time, y = pitch (on the ear-like mel scale), brightness = energy. `n_fft=1024` is the analysis window (64 ms), `hop_length=256` moves it 16 ms each step, `n_mels=64` frequency bands; `power_to_db` converts to decibels (log scale), which matches how we hear loudness. This figure is for **understanding**; the model computes its own version inside the network.

Augmentation

`augment`

```
def augment(y, rng):
```

`rng` is a random-number generator passed in so each call is independent.

```
    if rng.random() < 0.5:                  # random gain
        y = y * rng.uniform(0.5, 1.5)
```

With 50 % probability multiply the volume by a random factor between 0.5× and 1.5× (microphones record at different levels).

```
    if rng.random() < 0.5:                  # white Gaussian noise at a random SNR
        snr_db = rng.uniform(15, 35)
        y = y + rng.normal(0, 1, len(y)) * np.sqrt((np.mean(y ** 2) + 1e-10) / 10 ** (snr_db / 10))
```

With 50 % probability add random noise. The noise level is set by a *signal-to-noise ratio* between 15 and 35 dB: noise power = signal power / 10^(SNR/10). The `1e-10` prevents division problems for silent input.

```
    if rng.random() < 0.3:                  # pitch shift +-2 semitones
        y = librosa.effects.pitch_shift(y, sr=SR, n_steps=rng.uniform(-2, 2))
```

30 %: raise or lower the pitch by up to two semitones without changing speed (simulates different speakers).

```
    if rng.random() < 0.3:                  # time stretch 0.9-1.1x
        y = librosa.effects.time_stretch(y, rate=rng.uniform(0.9, 1.1))
```

30 %: speak 10 % faster or slower without changing pitch.

```
    return _fix_len(np.clip(y, -1, 1)).astype(np.float32)
```

Time stretching changes the length, so `_fix_len` crops/pads back to 48 000 samples; clipping keeps values in [−1, 1].

Why these probabilities? Pitch shift and time stretch are the most "aggressive" (and slowest) changes, so they are applied less often (30 %) than gain and noise (50 %).

`WaveDataset` and `make_loader`

```
class WaveDataset(torch.utils.data.Dataset):
```

PyTorch's standard way to feed data: a class that knows its length and can return item number *k*.

```
    def __init__(self, idx, augment_on=False):
        self.idx, self.augment_on = np.asarray(idx), augment_on
```

`idx` = the row numbers this dataset is allowed to use (e.g. only training rows). `augment_on` says whether to augment.

```
    def __len__(self): return len(self.idx)
```

Tells the `DataLoader` how many items exist (needed to know how many batches make an epoch).

```
    def __getitem__(self, k):
        j = self.idx[k]
        y = WAVES[j].astype(np.float32) / 32768.0
```

Look up the real row `j`, read its int16 samples and convert back to floats in [−1, 1].

```
        if self.augment_on:
            y = augment(y, np.random.default_rng())        # fresh randomness per call (safe with workers)
        return torch.from_numpy(y), int(LABELS[j])
```

Augment only for training; every call uses a **new** generator so parallel worker processes don't produce identical "random" changes. Return the audio tensor and its label.

```
def make_loader(idx, batch_size, train):
    return torch.utils.data.DataLoader(WaveDataset(idx, augment_on=train), batch_size=batch_size, shuffle=train,
                                       num_workers=NUM_WORKERS, pin_memory=(DEVICE.type == "cuda"),
                                       persistent_workers=NUM_WORKERS > 0)
```

A `DataLoader` groups items into batches. `shuffle=train` reshuffles only the training data each epoch; `num_workers` augments in background processes while the GPU is busy; `pin_memory` speeds up CPU→GPU copies; `persistent_workers` keeps the workers alive between epochs. **`augment_on=train`** is the line that guarantees *validation and test data are never augmented*.

Things to tell the mentor

- **Why augmenting after the split matters:** it is the single biggest data-leakage trap in tutorials; the notebook avoids it by design.
- **Train/serve parity:** `to_fixed` is shared with the API, so the app is judged by the same preprocessing as the experiments.
- **int16 + memory-mapping** is an engineering choice that makes a 12 k-clip dataset fit comfortably into free-tier RAM.

1c. The two model families (notebook §6)

Purpose

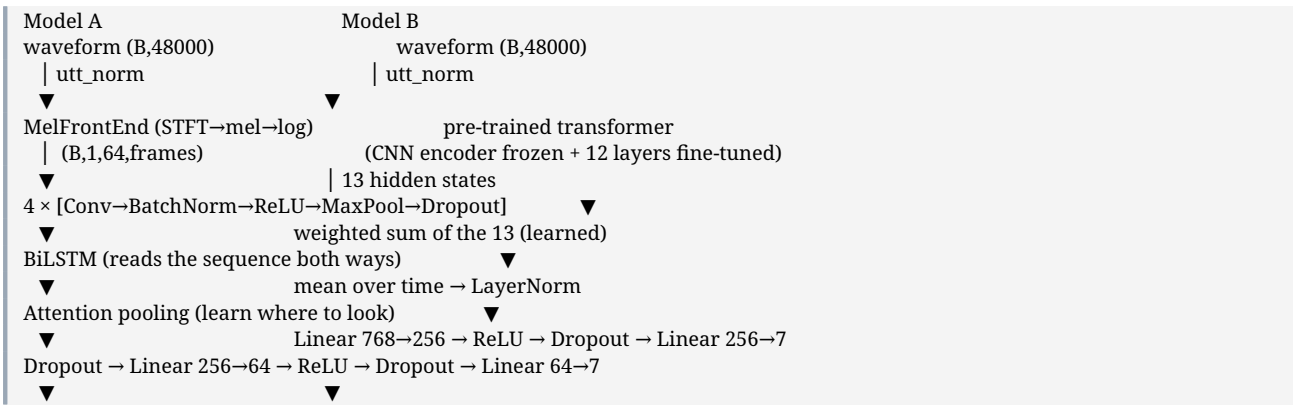
Define two kinds of classifier that both take a **3-second waveform `(batch, 48000)`** and return **7 scores (logits)**:

- **Model A – CNN + BiLSTM + Attention** on a mel spectrogram: small, quick, deployable anywhere.
- **Model B – self-supervised transformer** (Wav2Vec2, HuBERT or WavLM) fine-tuned for emotion: large, most accurate.

Why this component

Piece	What	Why	Trade-off
Model A baseline	≈ 0.95 M trainable parameters (+1.1 M fixed spectrogram constants), 8 MB as ONNX	Fits any free server; shows what a classical design achieves	Lower accuracy than transformers
Model B transformers	~95 M parameters, pre-trained on unlabeled speech	Already "understands" speech → small labelled data is enough	Big: needs int8 + a small server budget
Mel front-end *inside* the model	Spectrogram computed with convolutions	The exported ONNX file takes raw audio, so the API needs no spectrogram code	Slightly unusual implementation
Same input/output for both	`(B, 48000)` → `(B, 7)`	One training loop, one export function, one API	–
Weighted sum of layers	The model learns which transformer layers to use	Emotion lives in lower/middle layers, not only the last	Few extra parameters
Three backbones	Wav2Vec2, HuBERT, WavLM	Compare pre-training methods fairly	3× the training time
Frozen CNN feature encoder	Early audio convolutions not trained	Standard practice: stable, saves memory	Slightly less adaptation

Diagram



```
7 logits                    7 logits
```

Line-by-line walkthrough

`draw_pipeline` — the system diagram figure

```
def draw_pipeline():
    """Figure: end-to-end system diagram drawn with matplotlib (no external assets)."""
```

A helper that **draws the end-to-end system diagram** (microphone → upload → trim → transformer → softmax → average) with Matplotlib rectangles and arrows and saves it as `fig04_system_pipeline.png` via `show()`. It needs no image files, so the notebook stays self-contained; the figure is meant for the report.

`utt_norm` — shared by both models

```
def utt_norm(x):
    x = x - x.mean(dim=1, keepdim=True)
    return x / torch.sqrt((x ** 2).mean(dim=1, keepdim=True) + 1e-7)
```

For every clip: subtract its average (remove DC offset), then divide by its RMS (root-mean-square loudness). Result: zero mean and unit loudness. **Why:** different microphones record at different levels; the model should respond to *how* someone speaks, not how loud the phone is. The `1e-7` avoids dividing by zero for silence. Because this is *inside* the model, it also runs inside the deployed ONNX file.

`MelFrontEnd` — a spectrogram built from convolutions

```
class MelFrontEnd(nn.Module):
    def __init__(self, sr=SR, n_fft=1024, hop=256, n_mels=64, f_min=20., f_max=8000.):
        super().__init__()
        self.n_fft, self.hop = n_fft, hop
        win = torch.hann_window(n_fft)
```

Settings: 1024-sample analysis window (64 ms), 256-sample step (16 ms), 64 mel bands covering 20 Hz–8 kHz (speech lives below 8 kHz). A **Hann window** smoothly fades each analysis window in and out to avoid artefacts at the edges.

```
        k = torch.arange(n_fft // 2 + 1).unsqueeze(1).float(); n = torch.arange(n_fft).unsqueeze(0).float()
        ang = 2 * math.pi * k * n / n_fft
        self.register_buffer("cos_k", (torch.cos(ang) * win).unsqueeze(1))      # (F,1,n_fft)
        self.register_buffer("sin_k", (-torch.sin(ang) * win).unsqueeze(1))
```

The Fourier transform measures "how much of frequency *k* is in this window" by multiplying with cosine and sine waves. We pre-compute those waves (`cos_k`, `sin_k`) as **fixed convolution kernels**. `register_buffer` stores them with the model (saved and exported) but they are **not trainable**.

```
        fb = torchaudio.functional.melscale_fbanks(n_fft // 2 + 1, f_min, f_max, n_mels, sr, norm="slaney", mel_scale="slaney")
        self.register_buffer("mel_fb", fb)                                      # (F, n_mels)
```

The **mel filter-bank**: a matrix that merges the 513 linear frequency bins into 64 ear-like bands.

```
    def forward(self, x):                                                       # x: (B, T)
        x = F.pad(x.unsqueeze(1), (self.n_fft // 2, self.n_fft // 2), mode="reflect")
```

Add a channel dimension and mirror-pad the edges so the first and last windows are centred properly.

```
        re_, im_ = F.conv1d(x, self.cos_k, stride=self.hop), F.conv1d(x, self.sin_k, stride=self.hop)
        power = re_ ** 2 + im_ ** 2                                  # |STFT|^2   (B,F,frames)
```

Convolve with the cosine/sine kernels, stepping 256 samples at a time = the **real and imaginary parts of the STFT**. Squaring and adding gives the **power** (energy) at each frequency and time.

```
        mel = torch.matmul(power.transpose(1, 2), self.mel_fb).transpose(1, 2)    # (B, n_mels, frames)
        lm = torch.log(mel + 1e-6)
```

Apply the mel filter-bank (matrix multiply) and take the logarithm (human loudness perception is logarithmic; `1e-6` avoids `log(0)`).

```
        lm = (lm - lm.mean(dim=(1, 2), keepdim=True)) / torch.sqrt(lm.var(dim=(1, 2), keepdim=True, unbiased=False) + 1e-5)
        return lm.unsqueeze(1)                                       # (B, 1, n_mels, frames)
```

Standardise each spectrogram (mean 0, variance 1) and add a channel dimension so it looks like a one-channel image for the CNN.

Why not `torchaudio.transforms.MelSpectrogram`? Its FFT operator is hard to export to ONNX. Convolutions and matrix multiplications export cleanly, so the **whole model, including feature extraction, becomes one ONNX file**.

`CNNBiLSTMAttention` — Model A

```
        self.freq_mask = torchaudio.transforms.FrequencyMasking(freq_mask_param=10, iid_masks=True)   # SpecAugment
        self.time_mask = torchaudio.transforms.TimeMasking(time_mask_param=20, iid_masks=True)
```

SpecAugment: during training randomly hide a stripe of frequencies (up to 10 bands) and a stripe of time (up to 20 frames) in each spectrogram. It forces the model not to rely on any single region and fights over-fitting.

```
        def block(cin, cout, pool):
            return nn.Sequential(nn.Conv2d(cin, cout, 3, padding=1, bias=False), nn.BatchNorm2d(cout), nn.ReLU(inplace=True),
                                 nn.MaxPool2d(pool), nn.Dropout2d(0.1))
```

One CNN block: **Conv2d** (3×3 filters detect local patterns such as pitch movements) → **BatchNorm** (stabilises training; `bias=False` because BatchNorm adds its own offset) → **ReLU** (non-linearity) → **MaxPool** (keeps the strongest response, halves the size) → **Dropout2d** (randomly drops whole feature maps to reduce over-fitting).

```
        self.cnn = nn.Sequential(block(1, 32, (2, 2)), block(32, 64, (2, 2)), block(64, 128, (2, 1)), block(128, 128, (2, 1)))
```

Four blocks with more and more filters (32 → 64 → 128 → 128). The first two pool both frequency and time; the last two pool only frequency, so the **time resolution is kept for the LSTM**. After the four blocks the 64 mel bands are reduced by $2^4 = 16 \rightarrow 4$ bands.

```
        self.lstm = nn.LSTM(128 * (n_mels // 16), hidden, batch_first=True, bidirectional=True)
```

The **BiLSTM** reads the frame sequence forwards *and* backwards. Input size = 128 channels × 4 bands = 512 values per frame; `hidden=128` per direction → 256 outputs per frame. **Why:** emotion shows in *how the voice changes over time* (rising pitch, speeding up), which a recurrent layer captures.

```
        self.att = nn.Sequential(nn.Linear(2 * hidden, 128), nn.Tanh(), nn.Linear(128, 1))          # additive attention
```

A tiny network that gives each frame a score "how important is this moment".

```
        self.head = nn.Sequential(nn.Dropout(dropout), nn.Linear(2 * hidden, 64), nn.ReLU(), nn.Dropout(dropout), nn.Linear(64,
    n_classes))
```

The classifier: dropout → 256→64 → ReLU → dropout → 64→7 logits.

```
    def forward(self, wav):
        x = self.frontend(utt_norm(wav))                    # (B,1,M,T)
        if self.training:                                   # SpecAugment (training only)
            x = self.time_mask(self.freq_mask(x))
        x = self.cnn(x)                                     # (B,C,M/16,T/4)
```

Normalise → spectrogram → (training only) mask → CNN. `self.training` is `True` after `model.train()` and `False` after `model.eval()`, so masking never happens at test or inference time.

```
        b, c, f, t = x.shape
        h, _ = self.lstm(x.permute(0, 3, 1, 2).reshape(b, t, c * f))   # (B,T',2H)
```

Rearrange from (batch, channels, freq, time) to (batch, time, channels×freq) – the sequence layout the LSTM expects – and run it.

```
        w = torch.softmax(self.att(h), dim=1)               # attention weights over time
        return self.head((w * h).sum(dim=1))
```

Score every time step, `softmax` turns scores into weights that add up to 1, multiply each frame by its weight and **sum over time** to one 256-number summary of the whole clip; the head turns that into 7 logits.

```
    def param_groups(self):
        return [{"params": [p for p in self.parameters() if p.requires_grad], "lr": LR_A}]
```

Tells the optimiser which parameters to train and with which learning rate (one group here).

`SSLClassifier` — Model B

```
from transformers import AutoModel
```

Hugging Face's loader: gives the right class (Wav2Vec2Model, HubertModel, WavLMModel) for a model name.

```
        kw = dict(layerdrop=0.0, mask_time_prob=0.05, mask_time_length=5)     # light time-masking acts as regularisation
        if attn: kw["attn_implementation"] = attn                             # "eager" is the ONNX-friendly implementation
        self.backbone = AutoModel.from_pretrained(HF_IDS[name], **kw)
```

Download the pre-trained model. `layerdrop=0` keeps all layers active (stable); `mask_time_prob=0.05` randomly hides ~5 % of time steps in training (a mild regulariser built into these models). When exporting we pass `attn="eager"`, the attention implementation that converts to ONNX reliably.

```
        for prm in self.backbone.feature_extractor.parameters():            # the low-level CNN feature encoder stays frozen
            prm.requires_grad = False                                       # (done by hand: not every backbone class offers freeze_feature_encoder)
```

Freeze the convolutional feature encoder (the first stage that turns raw audio into frames). `requires_grad=False` means "do not update". *Why:* these low-level filters are generic; freezing is the standard recipe, saves memory and makes training stable.

```
        d, n_layers = self.backbone.config.hidden_size, self.backbone.config.num_hidden_layers + 1
        self.layer_w = nn.Parameter(torch.zeros(n_layers))                  # a_l (softmax -> weights); starts uniform
```

`d = 768` (feature size), `n_layers = 12 + 1 = 13` hidden states (the output of the embedding stage plus each of the 12 transformer layers). `layer_w` is a learnable vector of 13 numbers, initialised to zeros → after `softmax` every layer starts with the same weight 1/13.

```
        self.norm = nn.LayerNorm(d)
        self.head = nn.Sequential(nn.Dropout(0.2), nn.Linear(d, 256), nn.ReLU(), nn.Dropout(0.2), nn.Linear(256, n_classes))
```

LayerNorm stabilises the pooled features; the head maps 768 → 256 → 7.

```
    def forward(self, wav):
        hs = self.backbone(utt_norm(wav), output_hidden_states=True).hidden_states     # tuple of (B,T',d)
```

Run the transformer and ask it to return **all** intermediate layers, not just the last.

```
        w = torch.softmax(self.layer_w, dim=0).view(-1, 1, 1, 1)
        h = (w * torch.stack(hs, dim=0)).sum(dim=0)                    # weighted sum of layers
        return self.head(self.norm(h.mean(dim=1)))                     # mean-pool over time
```

Turn the 13 learnable numbers into weights summing to 1; stack the 13 layers into one tensor, multiply each by its weight and add them (the **learned layer mix**); average over time to a single 768-vector per clip; normalise; classify. *Why mix layers:* research on speech models shows that lower/middle layers keep prosody and voice quality — exactly what emotion needs — while the top layers become more about words.

```
    def param_groups(self):
        back = [p for p in self.backbone.parameters() if p.requires_grad]
        rest = [self.layer_w] + list(self.norm.parameters()) + list(self.head.parameters())
        return [{"params": back, "lr": LR_B_BACKBONE}, {"params": rest, "lr": LR_B_HEAD}]
```

Two learning rates: a tiny one (5e-5) for the pre-trained body, a larger one (1e-3) for the new parts. This is called *discriminative learning rates* and is a standard fine-tuning technique.

The registry and the summary table

```
ARCH = {"CNN-BiLSTM-Attn": dict(build=lambda: CNNBiLSTMAttention(), amp=False, bs=BATCH_A, ep1=P1_EPOCHS_A,
ep2=P2_EPOCHS_A, kind="cnn")}
for _n in HF_IDS:
    ARCH[_n] = dict(build=(lambda n=_n: SSLClassifier(n)), amp=True, bs=BATCH_B, ep1=P1_EPOCHS_B, ep2=P2_EPOCHS_B, kind="ssl")
```

A dictionary describing every model: how to build it (`build`), whether to use **mixed precision** (`amp`: 16-bit maths on the GPU for speed and memory — used for the transformers), batch size, epochs for each protocol, and the `kind` (used later when exporting). The `lambda n=_n` trick fixes the current loop value; without it all lambdas would build the *last* model.

```
    m = a["build"](); out = m.eval()(torch.randn(2, N_SAMPLES))
```

Build every model and push random audio through it: this checks that **all models accept `(2, 48000)` and output `(2, 7)`** before spending hours on training. The surrounding lines also print parameter counts (total and trainable).

Things to tell the mentor

- *Why two models?* The small one proves the idea is feasible on a tiny server; the large one shows what modern pre-trained speech models add.
- *Why not train a transformer from scratch?* It needs tens of thousands of hours of speech; fine-tuning a pre-trained one needs only our ~12 k clips.
- *Why is the mel front-end inside the network?* One file → one export → no feature code in the API → no train/serve mismatch.

1d. Training, the two evaluation protocols and the analysis (notebook §7–12)

Purpose

Train every model with **one shared training loop**, evaluate it in **two ways**, and understand *why* the numbers look the way they do.

Why this component

Piece	What	Why	Rejected alternative
One generic `train_model`	Same optimiser, loss, schedule for all models	Fair comparison	A custom loop per model
AdamW + One-Cycle schedule	Learning rate warms up, then decays	Fast, stable convergence for fine-tuning	Constant learning rate: slower, less stable
Class-weighted loss + label smoothing	Rare classes count more; targets are "soft"	Imbalance + more honest confidence	Plain cross-entropy
Gradient clipping	Limits the size of each update	Prevents rare exploding updates	–
Mixed precision (fp16)	Half-precision maths on GPU	~2× faster, less memory for transformers	fp32 only: slower, may not fit
Best-epoch checkpoint on **validation**	Keep the weights of the best validation epoch	Avoid over-fitted last epochs, no test peeking	Using the last epoch
Protocol 1 random split	80/10/10 stratified	Comparable with the literature	–
Protocol 2 GroupKFold	Speakers never shared between train and test	The real-user scenario	Reporting only Protocol 1
Per-dataset accuracy	Accuracy per corpus	Explains the combined score	Only one overall number

Diagram: the two protocols

```
PROTOCOL 1 – stratified random split            PROTOCOL 2 – 5-fold, speakers kept together
all clips (all speakers mixed)                  speakers:  A B C D E F G H I J …
  │ shuffle, keep emotion proportions             fold 1: test = {A,F}   train = everyone else
  ├── 80 % train   ← augmentation here only       fold 2: test = {B,G}   train = everyone else
  ├── 10 % validation ← pick the best epoch       …                      (no speaker in both sides)
  └── 10 % test    ← used once                  inside each training part: 10 % of the SPEAKERS
test speakers ALSO appear in train (leak!)        are held out as validation
```

Line-by-line walkthrough

`predict_probs` — run a model on a set of clips

```
@torch.no_grad()
def predict_probs(model, idx, bs=64, amp=False):
```

`@torch.no_grad()` switches off gradient bookkeeping (we are not training): less memory, faster.
`idx` = row numbers of the clips to score.

```
    model.eval().to(DEVICE)
```

`eval()` turns off dropout/masking and uses BatchNorm's running statistics; `.to(DEVICE)` moves the model to the GPU.

```
    for xb, _ in make_loader(idx, bs, train=False):
        with torch.autocast(DEVICE.type, enabled=amp and DEVICE.type == "cuda"):
            out.append(torch.softmax(model(xb.to(DEVICE)).float(), dim=-1).cpu())
    return torch.cat(out).numpy()
```

Loop over batches **without augmentation** (`train=False`), run the model, convert logits to probabilities with `softmax` (they add up to 1), move back to the CPU, and glue all batches into one array.

`train_model` — the training loop

```
    cfg = ARCH[arch]; seed_everything()
    model = cfg["build"]().to(DEVICE)
```

Look up the model description and build it fresh. Re-seeding before each model gives every run the same random start.

```
    cw = compute_class_weight("balanced", classes=np.arange(N_CLASSES), y=LABELS[train_idx])     # weights from the TRAIN fold only
    crit = nn.CrossEntropyLoss(weight=torch.tensor(cw, dtype=torch.float32, device=DEVICE), label_smoothing=0.05)
```

Class weights come from the **training clips only**; the **loss function** (`crit`) is cross-entropy ("how surprised is the model by the right answer") with those weights and **label smoothing 0.05** (the target "100 % angry" becomes "≈ 99 % angry, a little for the rest" so the model does not become over-confident).

```
    groups = model.param_groups()
    opt = torch.optim.AdamW(groups, weight_decay=1e-2)
```

AdamW is the optimiser (the algorithm that updates weights); it uses the parameter groups with their different learning rates; `weight_decay=0.01` gently pulls weights toward zero (regularisation).

```
    loader = make_loader(train_idx, cfg["bs"], train=True)
    sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=[g["lr"] for g in groups], epochs=epochs,
                                                steps_per_epoch=len(loader), pct_start=0.15)
```

The training loader **with augmentation**. **One-Cycle** raises the learning rate from small to the group's maximum during the first 15 % of training (`pct_start`), then lowers it smoothly to nearly zero. Warm-up protects the pre-trained weights from big early jolts; the decay lets the model settle.

```
    use_amp = cfg["amp"] and DEVICE.type == "cuda"
    scaler = torch.amp.GradScaler(enabled=use_amp)
```

Mixed precision only if the model asked for it and a GPU exists. The `GradScaler` multiplies the loss to avoid tiny fp16 gradients rounding to zero, then undoes it.

```
    best_acc, best_state, hist = -1, None, []
    for ep in range(epochs):
        model.train(); t0 = time.time(); tl = n = correct = 0
```

Start tracking the best validation accuracy and the history list. Each **epoch** = one pass over the training data; `model.train()` turns dropout/masking **on**.

```
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
```

```
        with torch.autocast(DEVICE.type, enabled=use_amp):
            logits = model(xb); loss = crit(logits.float(), yb)
```

For each batch: move data to the GPU; **forward pass** (the model's guess) under mixed precision; compute the loss against the true labels (in fp32 for accuracy).

```
        opt.zero_grad(set_to_none=True)
        scaler.scale(loss).backward(); scaler.unscale_(opt)
```

Clear old gradients; **backpropagation** (`backward`) computes how each weight should change; `unscale_` restores the true gradient size before clipping.

```
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(opt); scaler.update(); sched.step()
```

Clip the total gradient length to at most 1.0 (prevents rare huge steps), apply the weight update, update the scaler, and advance the learning-rate schedule by one step.

```
        tl += loss.item() * len(yb); n += len(yb); correct += (logits.argmax(1) == yb).sum().item()
```

Accumulate the running loss and the number of correct predictions for the epoch summary.

```
    va = accuracy_score(LABELS[val_idx], predict_probs(model, val_idx, amp=use_amp).argmax(1))
    hist.append(dict(epoch=ep + 1, train_loss=tl / n, train_acc=correct / n, val_acc=va))
```

After each epoch measure accuracy on the **validation** clips and store the history (used for the training-curve figure and for the report).

```
    if va > best_acc:
        best_acc = va; best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
model.load_state_dict(best_state)
return model, pd.DataFrame(hist)
```

If this epoch is the best so far, **copy the weights** (to CPU, detached) as a snapshot. At the end, load the best snapshot — so the returned model is the best-validation epoch, not simply the last one.

Helpers

```
def per_dataset_table(idx, pred):
    d = pd.DataFrame({"dataset": DATASETS[idx], "ok": (np.asarray(pred) == LABELS[idx])})
    t = d.groupby("dataset").ok.agg(["mean", "count"]); t.columns = ["accuracy", "n_clips"]
    return t
```

Mark each prediction right/wrong, group by source dataset and average: **accuracy per corpus**. This is what explains why the combined score is lower than the TESS/SAVEE scores.

```
    cm = confusion_matrix(y, p, labels=range(N_CLASSES), normalize="true")
```

(in `plot_cm`) A **confusion matrix**: row = true emotion, column = predicted emotion; `normalize="true"` shows percentages per row, so you can read "of all angry clips, how many were called sad?".

Protocol 1

```
all_idx = np.arange(len(df))
tr_idx, tmp_idx = train_test_split(all_idx, test_size=0.2, stratify=LABELS, random_state=SEED, shuffle=True)
va_idx, te_idx = train_test_split(tmp_idx, test_size=0.5, stratify=LABELS[tmp_idx], random_state=SEED, shuffle=True)
```

First cut: 80 % train / 20 % rest. Then split the 20 % half-and-half into **validation** and **test** (10 % each). `stratify=LABELS` keeps the emotion proportions the same in every part; `random_state` makes it repeatable.

```
shared = set(SPEAKERS[te_idx]) & set(SPEAKERS[tr_idx])
print(f"Leakage check: {len(shared)} of {len(set(SPEAKERS[te_idx]))} test speakers also appear in training (expected -> NOT speaker-independent)")
```

Counts the **speakers present in both train and test** — a deliberate, printed proof that Protocol 1 is *not* speaker-independent.

```
def run_protocol1(arch):
    if arch in RESULTS["p1"]:
        print(f"[{arch}] Protocol 1 already done (test acc {RESULTS['p1'][arch]['acc']*100:.2f}%) - skipping"); return
```

Resume logic: skip models that already finished.

```
    model, hist = train_model(arch, tr_idx, va_idx, ARCH[arch]["ep1"], tag=" P1")
    amp = ARCH[arch]["amp"]
    tp, vp = predict_probs(model, te_idx, amp=amp), predict_probs(model, va_idx, amp=amp)
    pred = tp.argmax(1)
```

Train on the 80 %, choose the best epoch on validation; then score the **test set once** (`tp`) and the validation set (`vp`, saved for the optional ensemble). `argmax(1)` = the emotion with the highest probability.

```
    RESULTS["p1"][arch] = dict(acc=float(accuracy_score(LABELS[te_idx], pred)), macro_f1=float(f1_score(LABELS[te_idx], pred, average="macro")),
                               val_acc=float(accuracy_score(LABELS[va_idx], vp.argmax(1))), n_test=len(te_idx), pred=pred.tolist(),
                               test_probs=tp.round(5).tolist(), val_probs=vp.round(5).tolist(), hist=hist.to_dict("records"))
    save_results()
```

Store accuracy, macro-F1, validation accuracy, all predictions, probabilities and the training history, then write the JSON file. Everything needed for the report figures is kept, so figures can be redrawn **without retraining**.

```
    del model; torch.cuda.empty_cache()
```

Free the GPU memory before the next model.

Optional ensemble (off by default)

```
    best = max((c for r in range(2, len(names) + 1) for c in itertools.combinations(names, r)),
               key=lambda c: accuracy_score(LABELS[va_idx], np.mean([np.array(RESULTS["p1"][a]["val_probs"]) for a in c], 0).argmax(1)))
```

Try every combination of two or more transformers, average their probabilities, and pick the combination with the best **validation** accuracy. Off by default because a deployed system should be one small model.

Protocol 2 — speaker-independent

```
def run_protocol2(arch):
    res = RESULTS["p2"].setdefault(arch, {"folds": []})
    done = {f["fold"] for f in res["folds"]}
    for k, (trn, tst) in enumerate(GroupKFold(n_splits=N_FOLDS).split(all_idx, LABELS, groups=SPEAKERS)):
```

`GroupKFold` makes 5 folds so that **all clips of a speaker go to the same side**. `groups=SPEAKERS` is the key argument. `done` lists finished folds for resuming.

```
        assert not (set(SPEAKERS[trn]) & set(SPEAKERS[tst])), "speaker leakage between train and test!"
```

A built-in **proof**: the set of speakers in training and in testing must be disjoint, otherwise the program stops.

```
    a, b = next(GroupShuffleSplit(n_splits=1, test_size=0.1, random_state=SEED).split(trn, groups=SPEAKERS[trn]))   # inner val = unseen speakers
    model, hist = train_model(arch, trn[a], trn[b], ARCH[arch]["ep2"], tag=f" P2 fold{k}")
```

Inside the training part, hold out about 10 % of the **speakers** (again by group) as the validation set, so that choosing the best epoch is also speaker-independent. The test fold is not touched during training.

```
    pred = predict_probs(model, tst, amp=ARCH[arch]["amp"]).argmax(1)
    acc = float(accuracy_score(LABELS[tst], pred))
```

Score the test fold — people the model has **never heard**.

```
    if acc >= max(f["acc"] for f in res["folds"]):        # keep the best fold's weights for deployment
        torch.save(model.state_dict(), os.path.join(WORK_DIR, f"best_p2_{arch}_{TAG}.pt")); res["best_fold"] = k
```

Save the weights of the best fold as a `.pt` file – **this is the model that later becomes the ONNX file for the app**.

```
accs = [f["acc"] for f in res["folds"]]
res["mean"], res["std"] = float(np.mean(accs)), float(np.std(accs)); save_results()
```

Mean and standard deviation over the 5 folds: "76 % ± 5" tells the reader both the typical accuracy and how much it varies between groups of speakers.

Choosing which transformer gets Protocol 2 (no test-set peeking)

```
ssl_ranked = sorted([a for a in HF_IDS if a in RESULTS["p1"]], key=lambda a: -RESULTS["p1"][a]["val_acc"])
P2_CHOSEN = ssl_ranked[:P2_TRANSFORMERS]
```

Rank the transformers by **validation** accuracy from Protocol 1 and run the expensive 5-fold only for the top one(s). This saves GPU hours and keeps the choice free of test-set information.

Analysis cells (§11)

- **Comparison table and bar chart** put Protocol 1 and Protocol 2 side by side with the gap (`P1 − P2`).
- **Error analysis:** builds the confusion matrix, lists the most frequent confusions (for example *neutral* ↔ *sad*) — they are the emotions that share pitch/energy patterns.
- **Layer-weights plot:** `torch.softmax(state["layer_w"], 0)` shows which transformer layers the best model preferred; weight on lower/middle layers supports the claim that emotion is mostly acoustic/prosodic information.
- **§12 final results:** builds a markdown table and a **ready-to-paste abstract** from the measured numbers, so no number in the summary is typed by hand.

Things to tell the mentor

- **Two protocols** = **intellectual honesty.** In the earlier Kaggle run the Wav2Vec2 model scored 86.5 % on the random split and 76.6 % on unseen speakers; the notebook is designed to show *both*.

- **Where leakage was prevented:** augmentation (train only), class weights (train fold only), model selection (validation only), speaker-disjoint folds (asserted).
- **Why CREMA-D limits the score:** 60 % of the clips, crowd-labelled, with ambiguous emotions; the per-dataset table makes this visible.

1e. Export, quantisation and real-time inference (notebook §13)

Purpose

Turn the best trained PyTorch model into a **single portable file** (`model.onnx`, optionally `model_int8.onnx`) that a tiny CPU server can run without PyTorch, check that **nothing was lost** in the conversion, and demonstrate **real-time** use with voice-activity detection.

Why this component

Piece	What	Why	Rejected alternative
Export from the best **Protocol-2** fold	The deployed model is the best speaker-independent model	It is the one that generalises to new people	Deploying the best random-split model
ONNX format	Portable graph of the network	Runs in ONNX Runtime without PyTorch	Shipping PyTorch (hundreds of MB)
`attn="eager"` for export	Plain attention implementation	The optimised implementation does not export reliably	Default attention: export errors
Dynamic batch axis	`(batch, 48000)` with variable batch	Server can send 1 or several windows	Fixed batch of 1
int8 quantisation of MatMul only	Weights stored as 8-bit	≈ 4× smaller, faster on CPU	Quantising convolutions too: hurts accuracy
Fidelity table (PyTorch vs ONNX vs int8)	Measures accuracy, agreement, size, latency on real unseen-speaker clips	Never assume a conversion is lossless	Trusting the converter
Fallback to the next candidate	If an export fails, try the next best model	A pipeline that survives one failure	Crashing after 6 hours
silero-VAD + sliding window	Speech detection + 3 s windows with 50 % overlap	Real-time use on streams; skip silence	Classifying silence
Zip bundle	Figures, results, model files in one download	Easy hand-over for the report/deployment	Collecting files one by one

Diagram

```
best_p2_<model>.pt  (PyTorch weights of the best unseen-speaker fold)
      │ build_for_export  →  same architecture, attention="eager"
      ▼
torch.onnx.export ──▶ model.onnx  (float32, ≈ 4 bytes/weight)
      ╵ quantize_dynamic(MatMul only, int8, per-channel)
      ▼
model_int8.onnx (≈ 1/4 size)
      ╵
      ├─ fidelity check on up to 300 real unseen-speaker clips:
      ╵    accuracy, agreement with PyTorch, size, 1-thread latency
      ▼
recommended file = int8 if accuracy loss < 1 point, else float32
      ▼
labels.json (class order, sample rate, window length, which file to serve)
```

Line-by-line walkthrough

Choosing and building the model

```
exp_candidates = sorted(p2_models, key=lambda a: -RESULTS["p2"][a]["mean"])
```

List all models that finished Protocol 2, **best speaker-independent mean first**.

```
def build_for_export(arch):
    return CNNBiLSTMAttention() if ARCH[arch]["kind"] == "cnn" else SSLClassifier(arch, attn="eager")
```

Create an empty copy of the same architecture. The CNN is the same class; transformers are built with `attn="eager"` (the simple attention maths that ONNX understands). The `kind` field from the `ARCH` registry decides which.

The export loop

```
EXPORTED = None
for arch in exp_candidates:                    # fall back to the next candidate if an export fails
    try:
        m = build_for_export(arch); m.load_state_dict(torch.load(os.path.join(WORK_DIR, f"best_p2_{arch}_{TAG}.pt"),
map_location="cpu")); m.eval().cpu()
```

For each candidate: build the model, **load the saved best-fold weights**, switch to evaluation mode (no dropout/masking) and move to the CPU (export is traced on CPU).

```
        kw = dict(input_names=["waveform"], output_names=["logits"], dynamic_axes={"waveform": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17)
```

Names for the input and output (the API uses `"waveform"`), a **dynamic batch dimension** (axis 0 may be any size), and **opset 17** – the version of the ONNX operator set (stable and supported by ONNX Runtime).

```
        try:    torch.onnx.export(m, (torch.randn(1, N_SAMPLES),), ONNX_PATH, dynamo=False, **kw)
        except TypeError: torch.onnx.export(m, (torch.randn(1, N_SAMPLES),), ONNX_PATH, **kw)
```

`torch.onnx.export` **traces** the model by running a dummy 3-second input through it and records every operation. `dynamo=False` selects the classic, reliable exporter (newer PyTorch versions have a new one that handles transformers less reliably); older PyTorch versions do not know that argument, hence the `TypeError` fallback.

```
        EXPORTED = arch; EXPORT_MODEL = m; print("exported:", arch, f"({os.path.getsize(ONNX_PATH)/1e6:.1f} MB)"); break
    except Exception as e:
        print(f"export of {arch} failed ({repr(e)[:160]}) -> trying next candidate")
assert EXPORTED, "no model could be exported"
```

On success remember which model was exported and stop; on failure print a short error and try the next candidate. If none works the notebook stops with a clear message.

Quantisation

```
try:
    quantize_dynamic(ONNX_PATH, INT8_PATH, weight_type=QuantType.QInt8, op_types_to_quantize=["MatMul"], per_channel=True)
except Exception as e:
    print("INT8 quantisation failed:", repr(e)[:200]); INT8_PATH = None
```

Dynamic quantisation stores weights as `int8` (a number between −128 and 127 plus a scale factor) and converts activations on the fly. Settings: only `MatMul` operations are quantised (these are the large linear layers of the transformer, where almost all weights are); `per_channel=True` uses one scale per output channel, which is more accurate than one scale for the whole matrix. If quantisation fails the notebook continues with the float32 file.

Fidelity check: is the exported model still the same model?

```
best_fold = RESULTS["p2"][EXPORTED]["best_fold"]
fold_rec = next(f for f in RESULTS["p2"][EXPORTED]["folds"] if f["fold"] == best_fold)
chk_idx = np.array(fold_rec["idx"])[:300]
```

Take the **test clips of the best fold** — speakers the exported model never saw — up to 300 of them.

```
def run_onnx(path, x, bs=8, threads=2):
    so = ort.SessionOptions(); so.intra_op_num_threads = threads
    s = ort.InferenceSession(path, so, providers=["CPUExecutionProvider"])
    out = np.concatenate([s.run(None, {"waveform": x[i:i + bs]})[0] for i in range(0, len(x), bs)])
```

Create an ONNX Runtime session on the CPU and score the clips in batches of 8. `s.run(None, {"waveform": ...})` is **exactly the call the API makes**.

```
    s1 = ort.SessionOptions(); s1.intra_op_num_threads = 1; s1 = ort.InferenceSession(path, s1, providers=["CPUExecutionProvider"])
    s1.run(None, {"waveform": x[:1]}); t0 = time.time(); [s1.run(None, {"waveform": x[:1]}) for _ in range(3)]
    return out, (time.time() - t0) / 3 * 1000
```

Measure **latency**: a second session with **one thread** (a stand-in for a small free server), one warm-up run (the first call is always slower), then the average of 3 timed runs in milliseconds.

```
with torch.no_grad(): ref_logits = np.concatenate([EXPORT_MODEL(torch.from_numpy(xb[i:i + 8])).numpy() for i in range(0, len(xb), 8)])
```

The **reference**: the original PyTorch model's outputs on the same clips.

```
    fid.append({"Variant": name, "Accuracy (%)": round((out.argmax(1) == yb).mean() * 100, 2), "Agreement with PyTorch (%)":
round((out.argmax(1) == ref_logits.argmax(1)).mean() * 100, 2),
                "Size (MB)": round(os.path.getsize(path) / 1e6, 1), "Latency 1 thread (ms)": round(lat)})
```

For each exported file: accuracy against the true labels, **agreement** with the PyTorch prediction (should be ≈ 100 % for float32), file size and latency. The table lets you state the quantisation trade-off with numbers.

The recommendation rule and `labels.json`

```
use_int8 = INT8_PATH is not None and "ONNX int8 (MatMul only)" in fid_df.index and \
        fid_df.loc["ONNX fp32", "Accuracy (%)"] - fid_df.loc["ONNX int8 (MatMul only)", "Accuracy (%)"] < 1.0
SERVE_PATH = INT8_PATH if use_int8 else ONNX_PATH
```

Use int8 only if it loses less than one accuracy point; otherwise stay with float32. The decision is automatic and based on measured data.

```
json.dump({"labels": EMOTIONS, "sample_rate": SR, "window_samples": N_SAMPLES, "top_db": TOP_DB, "architecture": EXPORTED,
        "protocol2_mean_acc": RESULTS["p2"][EXPORTED]["mean"], "protocol1_acc": RESULTS["p1"].get(EXPORTED, {}).get("acc"),
        "recommended_file": os.path.basename(SERVE_PATH)}, open(os.path.join(EXPORT_DIR, "labels.json"), "w"), indent=2)
```

`labels.json` is the **contract between the notebook and the API**: class order (so output index 3 means "happy"), sample rate, window length, trimming threshold, which architecture it is, and which file to serve. The backend reads it at start-up (`meta = json.load(...)`), so the API never hard-codes these values.

Real-time inference with VAD

```
from silero_vad import load_silero_vad, get_speech_timestamps
VAD = load_silero_vad()
SESS = ort.InferenceSession(SERVE_PATH, providers=["CPUExecutionProvider"])
```

silero-VAD is a small pre-trained neural network that finds *where someone is speaking*. SESS is the ONNX session of the model we are going to deploy — so the demo uses the real deployment file.

```
def classify_window(y):
    logits = SESS.run(None, {"waveform": to_fixed(y)[None]})[0][0]
    p = np.exp(logits - logits.max()); p /= p.sum()
    return EMOTIONS[int(p.argmax())], float(p.max()), p
```

to_fixed(y)[None] prepares one window with the shared preprocessing and adds the batch dimension. The two p lines are a hand-written **softmax** (subtracting the maximum first prevents overflow). Returns the emotion name, its probability (**confidence**) and all probabilities.

```
def stream_emotions(y, win_s=3.0, hop_s=1.5, min_speech=0.3, verbose=True):
    ts = get_speech_timestamps(torch.from_numpy(y), VAD, sampling_rate=SR)
    speech = np.zeros(len(y), dtype=bool)
    for t in ts: speech[t["start"]:t["end"]] = True
```

Ask the VAD for speech segments (start/end sample numbers) and mark those samples True in a yes/no array the same length as the audio.

```
    W, H, out = int(win_s * SR), int(hop_s * SR), []
    for s in range(0, max(1, len(y) - W // 2), H):
        seg, sp = y[s:s + W], speech[s:s + W]
```

W = window length in samples (3 s), H = hop (1.5 s = **50 % overlap**). Slide the window along the signal; seg is the audio, sp the speech marks inside it.

```
        if rec["speech_frac"] >= min_speech:
            t0 = time.time(); e, c, _ = classify_window(seg); rec.update(emotion=e, confidence=c, latency_ms=(time.time() - t0) * 1000)
```

Only windows in which at least **30 % is speech** are classified; silent windows are skipped (they would produce meaningless predictions) and the latency of each real prediction is recorded.

Why 50 % overlap? Emotion can change; overlapping windows give a new reading every 1.5 s and avoid missing a change that falls on a window border.

Bundle for hand-over

```
zip_path = shutil.make_archive(os.path.join(os.path.dirname(OUT_DIR), "ser_report_bundle"), "zip", OUT_DIR)
```

Packs the model files, figures, results_for_report.json and report_summary.md into one .zip that you download from Kaggle's *Output* tab.

Things to tell the mentor

- The **deployed model comes from Protocol 2**, i.e. it is chosen for generalisation to new speakers.
- **We measured the cost of compression** (accuracy, agreement, size, latency) before choosing the served file.
- labels.json documents **class order and preprocessing**, preventing a classic bug: a server that preprocesses differently from training.
- The real-time demo uses the **same ONNX file as the cloud API**.